Lab 7 – Programs on String in CUDA

Q1.cu : Write a program in CUDA to count the number of times a given word is repeated in a sentence. (Use Atomic function)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <cuda_runtime.h>

__global__ void countWord(char *sentence, char *word, int *count, int sLen, int
wLen) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i <= sLen - wLen) {
        bool match = true;
        for (int j = 0; j < wLen; j++) {
            if (sentence[i + j] != word[j]) {
                match = false;
                break;
            }
        }
        if (match) atomicAdd(count, 1);
    }
}

int main() {
    char sentence[1024], word[128];
    printf("Enter a sentence: ");
    fgets(sentence, sizeof(sentence), stdin);
    sentence[strcspn(sentence, "\n")] = '\0';

    printf("Enter the word to search: ");
    fgets(word, sizeof(word), stdin);
    word[strcspn(word, "\n")] = '\0';

    int sLen = strlen(sentence), wLen = strlen(word);

    for (int i = 0; i < sLen; i++) sentence[i] = tolower(sentence[i]);
    for (int i = 0; i < wLen; i++) word[i] = tolower(word[i]);

    char *d_sentence, *d_word;
    int *d_count, h_count = 0;
```

```
    cudaMalloc(&d_sentence, sLen);
    cudaMalloc(&d_word, wLen);
    cudaMalloc(&d_count, sizeof(int));

    cudaMemcpy(d_sentence, sentence, sLen, cudaMemcpyHostToDevice);
    cudaMemcpy(d_word, word, wLen, cudaMemcpyHostToDevice);
    cudaMemcpy(d_count, &h_count, sizeof(int), cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (sLen + threads - 1) / threads;

    countWord<<<blocks, threads>>>(d_sentence, d_word, d_count, sLen, wLen);
    cudaMemcpy(&h_count, d_count, sizeof(int), cudaMemcpyDeviceToHost);
    printf("The word '%s' appears %d times in the sentence.\n", word, h_count);

    cudaFree(d_sentence);
    cudaFree(d_word);
    cudaFree(d_count);
    return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L7$ nvcc -o abc q1.cu
6CSEC1@debian:~/Documents/230905152_PPL/L7$ ./abc
Enter a sentence: hi bye hi hi bye
Enter the word to search: hi
The word 'hi' appears 3 times in the sentence.
```

Q2.cu :

Write a CUDA program that reads a string § and produces the string RS as follows:

Input string §: PCAP

Output string RS: PCAPPCAPCP

Note: Each work item copies required number of characters from S in RS.

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>
```

```
__global__ void buildRS(char *S, char *RS, int lenS) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int lenRS = (lenS * (lenS + 1)) / 2;
    if (i < lenRS) {
        int pos = i;
        for (int k = lenS; k > 0; k--) {
            if (pos < k) {
                RS[i] = S[pos];
                break;
            }
            pos -= k;
        }
    }
}

int main() {
    char h_S[128];
    printf("Enter string S: ");
    fgets(h_S, sizeof(h_S), stdin);
    h_S[strcspn(h_S, "\n")] = '\0';

    int lenS = strlen(h_S);
    int lenRS = (lenS * (lenS + 1)) / 2;

    char *d_S, *d_RS;
    char h_RS[1024];

    cudaMalloc(&d_S, lenS);
    cudaMalloc(&d_RS, lenRS);

    cudaMemcpy(d_S, h_S, lenS, cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (lenRS + threads - 1) / threads;
    buildRS<<<blocks, threads>>>(d_S, d_RS, lenS);

    cudaMemcpy(h_RS, d_RS, lenRS, cudaMemcpyDeviceToHost);
    h_RS[lenRS] = '\0';

    printf("Output string RS: %s\n", h_RS);

    cudaFree(d_S);
```

```
    cudaFree(d_RS);
    return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L7$ nvcc -o abc q2.cu
6CSEC1@debian:~/Documents/230905152_PPL/L7$ ./abc
Enter string S: 12345
Output string RS: 123451234123121
```

A1.cu : Write a CUDA program which reads a string consisting of N words and reverse each word of it in parallel.

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>
__global__ void reverseWords(char *in, char *out, int *starts, int *lens, int wc)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < wc) {
        int s = starts[i];
        int l = lens[i];
        for (int j = 0; j < l; j++) {
            out[s + j] = in[s + l - 1 - j];
        }
    }
}

int main() {
    char input[256];
    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';
    int n = strlen(input);

    int starts[50], lens[50], wc = 0;
    for (int i = 0; i < n;) {
        while (i < n && input[i] == ' ') i++;
        if (i >= n) break;
        int s = i;
        while (i < n && input[i] != ' ') i++;
        starts[wc] = s;
        lens[wc] = i - s;
        wc++;
```

```
    }

    char *d_in, *d_out;
    int *d_s, *d_l;
    cudaMalloc(&d_in, n);
    cudaMalloc(&d_out, n);
    cudaMalloc(&d_s, wc * sizeof(int));
    cudaMalloc(&d_l, wc * sizeof(int));

    cudaMemcpy(d_in, input, n, cudaMemcpyHostToDevice);
    cudaMemcpy(d_s, starts, wc * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_l, lens, wc * sizeof(int), cudaMemcpyHostToDevice);

    reverseWords<<<(wc + 255) / 256, 256>>>(d_in, d_out, d_s, d_l, wc);

    char output[256];
    cudaMemcpy(output, d_out, n, cudaMemcpyDeviceToHost);
    for (int i = 0; i < n; i++) {
        if (input[i] == ' ') output[i] = ' ';
    }
    output[n] = '\0';
    printf("Result: %s\n", output);

    cudaFree(d_in);
    cudaFree(d_out);
    cudaFree(d_s);
    cudaFree(d_l);
    return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L7$ nvcc -o abc a1.cu
6CSEC1@debian:~/Documents/230905152_PPL/L7$ ./abc
Enter a string: this is ppl lab 7
Result: siht si lpp bal 7
```

A2.cu : Write a CUDA program that takes a string Sin as input and one integer value N and produces an output string , Sout, in parallel by concatenating input string Sin, N times as shown below.

Input: Sin = “Hello” and N=3

Ouput: Sout = “HelloHelloHello”

Note: Every thread copies the same character from the Input string S, N times to the re-quired position.

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>
__global__ void concatKernel(char *in, char *out, int len, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int total = len * N;
    if (idx < total) {
        out[idx] = in[idx % len];
    }
}

int main() {
    char Sin[256];
    int N;
    printf("Input: Sin: ");
    fgets(Sin, sizeof(Sin), stdin);
    Sin[strcspn(Sin, "\n")] = '\0';
    printf("Enter N: ");
    scanf("%d", &N);

    int len = strlen(Sin);
    int totalLen = len * N;

    char *d_in, *d_out;
    cudaMalloc(&d_in, len);
    cudaMalloc(&d_out, totalLen);
    cudaMemcpy(d_in, Sin, len, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocks = (totalLen + threadsPerBlock - 1) / threadsPerBlock;
    concatKernel<<<blocks, threadsPerBlock>>>(d_in, d_out, len, N);

    char Sout[1024];
    cudaMemcpy(Sout, d_out, totalLen, cudaMemcpyDeviceToHost);
    Sout[totalLen] = '\0';

    printf("Output: Sout: %s\n", Sout);
    cudaFree(d_in);
    cudaFree(d_out);
    return 0;
```

```
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L7$ nvcc -o abc a2.cu
6CSEC1@debian:~/Documents/230905152_PPL/L7$ ./abc
Input:      Sin: Cos
Enter N: 4
Output:     Sout: CosCosCosCos
```

A3.cu : Write a CUDA program which reads a string Sin and produces an output string T as shown below.

Input: Sin: “Hai”

Ouput: T: “Haait “

Note:Every thread stores a character from input string Sin, required number of times into output string.

```
#include <stdio.h>
#include <string.h>
#include <cuda_runtime.h>
__global__ void repeatChars(char *in, char *out, int *pos, int len) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < len) {
        char c = in[idx];
        int start = pos[idx];
        int count = idx + 1;
        for (int j = 0; j < count; j++) {
            out[start + j] = c;
        }
    }
}

int main() {
    char Sin[256];
    printf("Input Sin: ");
    fgets(Sin, sizeof(Sin), stdin);
    Sin[strcspn(Sin, "\n")] = '\0';
    int len = strlen(Sin);

    int pos[256];
    int totalLen = 0;
    for (int i = 0; i < len; i++) {
```

```
        pos[i] = totalLen;
        totalLen += (i + 1);
    }

    char *d_in, *d_out;
    int *d_pos;
    cudaMalloc(&d_in, len);
    cudaMalloc(&d_out, totalLen);
    cudaMalloc(&d_pos, len * sizeof(int));

    cudaMemcpy(d_in, Sin, len, cudaMemcpyHostToDevice);
    cudaMemcpy(d_pos, pos, len * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocks = (len + threadsPerBlock - 1) / threadsPerBlock;
    repeatChars<<<blocks, threadsPerBlock>>>(d_in, d_out, d_pos, len);

    char T[1024];
    cudaMemcpy(T, d_out, totalLen, cudaMemcpyDeviceToHost);
    T[totalLen] = '\0';

    printf("Output: T: %s\n", T);

    cudaFree(d_in);
    cudaFree(d_out);
    cudaFree(d_pos);
    return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L7$ nvcc -o abc a3.cu
6CSEC1@debian:~/Documents/230905152_PPL/L7$ ./abc
Input      Sin: abcde
Output:      T: abbcccddddeeeee
```